Vulnerability Assessment Report

Internship program: Future Intern – Cybersecurity

Task: **Task 1**

Prepared by: **Remi Eli Kokou AKAKPO**

Role: **Cybersecurity intern**

Date: **04/03/2026**

1. Introduction

This report presents the results of a security assessment performed on the website **demo.testfire.net**.
The test was performed on March 4, 2025.
The analysis was performed using a “Black Box” approach (without prior access to the source code), using recognized standards such as the OWASP Top 10.

2. Objective

The main objective of this audit mission is to assess the security posture of the **demo.testfire.net** website by combining a vulnerability scan with OWASP ZAP, and an in-depth manual inspection using Browser DevTools to validate the configuration of HTTP headers, cookie protection, and the integrity of client-side exchanges.

3. Scope

This section defines the scope and boundaries of the project.

Website Name	Altoro Mutual
URL	http://altoro.testfire.net/

3.1. Assessment Attribute(s)

Parameter	Value
Starting Vector	External
Target Criticality	Critical
Assessment Nature	Cautious & Calculated
Assessment Conspicuity	Clear
Proof of Concept(s)	Attached wherever possible and applicable.

3.2. Risk Calculation and Classification

Following is the risk classification:

Info	Low	Medium	High	Critical
No direct threat to host/ individual user account. Sensitive information can be revealed to the adversary.	Vulnerabilities may not have public exploit (code) available or cannot be exploited in the wild. Vulnerability observed may not have high rate of occurrence. Patch workaround released by vendor.	Vulnerabilities may not have public exploit (code) available or cannot be exploited in the wild. Patch/ workaround not yet released by vendor.	Vulnerabilities which can be exploited publicly, workaround or fix/ patch available by vendor.	Vulnerabilities which can be exploited publicly, workaround or fix/ patch may not be available by vendor.

Table 1: Risk Rating

Summary

Here is an application security assessment in "Black Box" mode performed on the **Altoro Mutual** web application.

http://altoro.testfire.net/

Following section illustrates **Detailed** Technical information about identified vulnerabilities.

Total: 8 Vulnerabilities

High	Medium	Low
0	**5**	**3**

1. Absence of anti-CSRF tokens

Reference No:	Risk Rating:
WEB_VUL_01	**Medium**

Tools Used:

OWASP ZAP

Vulnerability Description:

The website does not verify whether forms (submit buttons, transfers, password changes) are actually validated by the user or by a hidden hacker.

An analogy to understand CSRF

Imagine you are at the bank and you have just identified yourself at the counter. The banker knows you and leaves his session open. A hacker, hidden behind you, slips a pre-filled piece of paper into your hand that says: "Transfer €1,000 to the hacker's account." You sign without looking because you are distracted. The banker sees your signature and validates the transaction.

CSRF is exactly that: the hacker takes advantage of the fact that you are already logged in to the site to get you to approve an action without your knowledge.

Vulnerability Identified by / How It Was Discovered

Manual Analysis & Automated Analysis

Vulnerable URLs / IP Address

https://demo.testfire.net/feedback.jsp

Implications / Consequences of not Fixing the Issue

- A hacker can force a logged-in user to modify their personal information, change their password, or delete their account without their knowledge.
- The absence of Anti-CSRF tokens would allow an attacker to trigger fund transfers to a third-party account without the user clicking "Validate."
- Without a strict CSP policy, a script injection (XSS) could allow session cookies to be stolen, giving the hacker full access to the victim's account.

Suggested Countermeasures

To correct the lack of protection against cross-site request forgery, the following measures must be adopted:

- Generate a unique, random, and unpredictable token for each session or form. This token must be verified by the server before validating any user action.
- Favor recognized libraries such as OWASP CSRFGuard or session management via ESAPI to automate this protection.
- For sensitive operations (money transfers…), enforce an additional confirmation step or re-authentication.
- Never use the GET method for actions that modify data (such as deleting an account or changing a password). Use only POST, PUT, or DELETE.
- Check the origin of the request via the HTTP header to ensure that it comes from your own domain (although this method is less robust than tokens).
- Protect against XSS

It is imperative to note that all anti-CSRF protections can be bypassed if the site has an XSS vulnerability. A hacker could use an injected script to read the secret token and use it. Security must therefore be comprehensive.
References
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html https://cwe.mitre.org/data/definitions/352.html

Proof of concept:

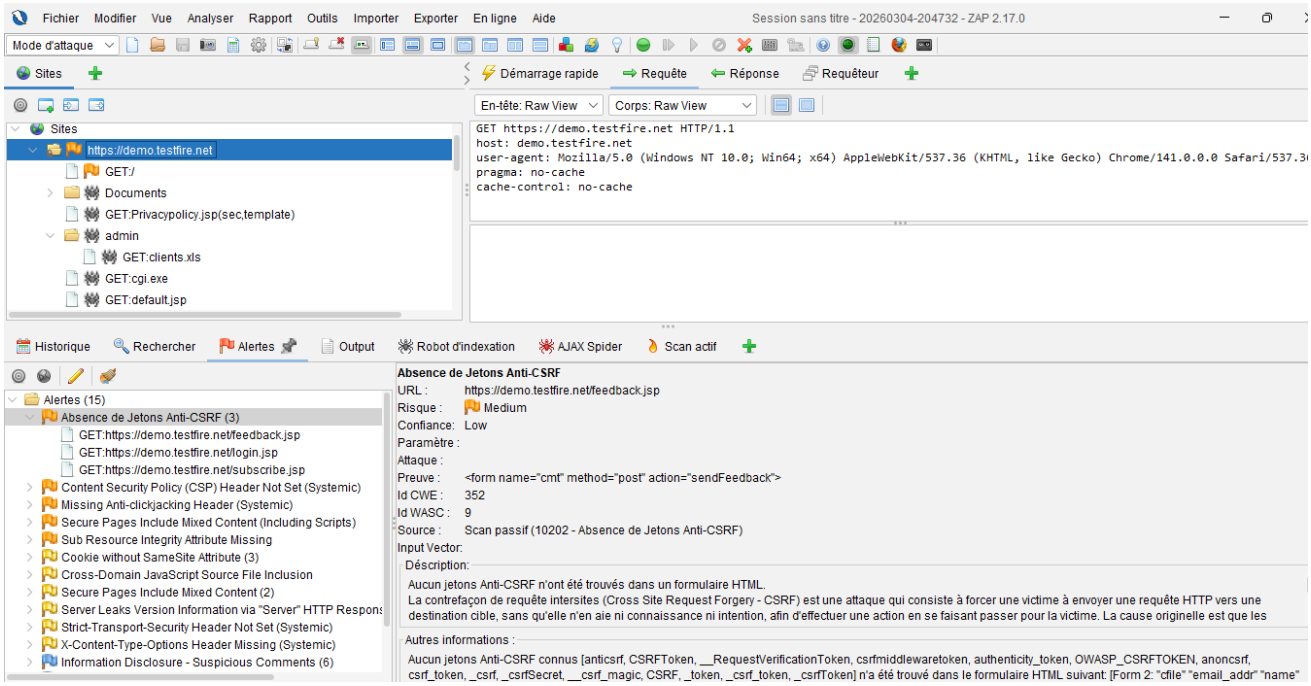


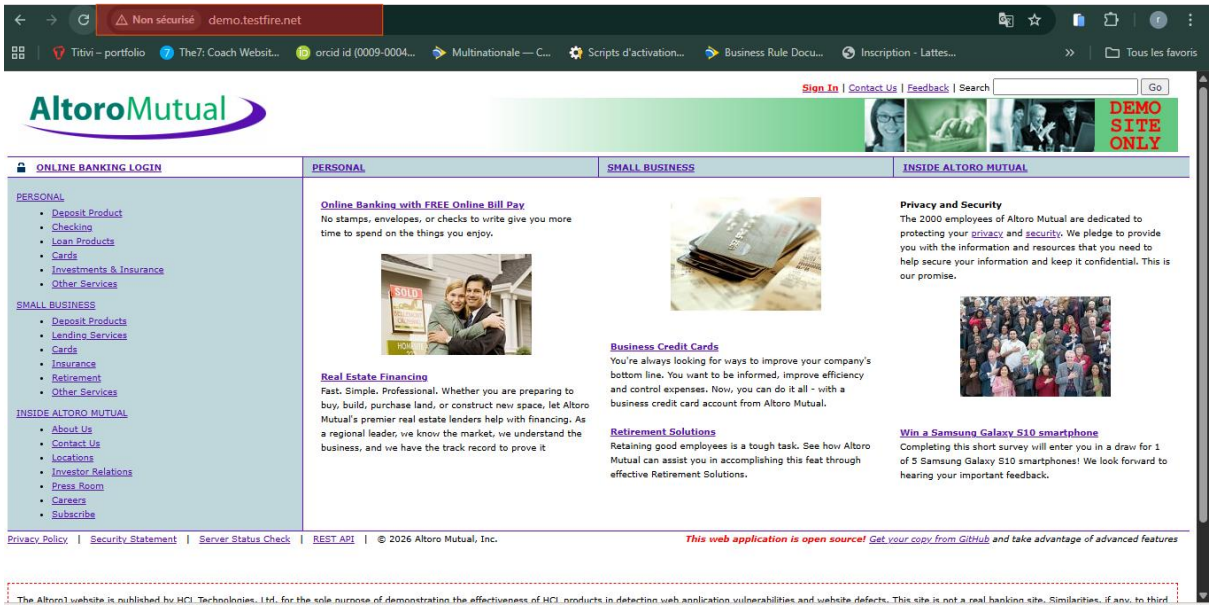
Fig 1: Absence of anti-CSRF tokens

2. Content Security Policy (CSP) Header Not Set

Reference No:	**Risk Rating:**
WEB_VUL_02	**Medium**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
The application does not have a “content security policy.” This is a missing safeguard that leaves the user's web browser without instructions on what it is allowed to execute and what it is not.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis	
Vulnerable URLs / IP Address	
https://demo.testfire.net/	

Implications / Consequences of not Fixing the Issue
• The absence of the Content Security Policy (CSP) header exposes the Altoro Mutual application to injection attacks, including Cross-Site Scripting (XSS) attacks. Without this directive, the client's browser cannot distinguish between legitimate scripts from the site and malicious scripts inserted by an attacker, which could lead to the theft of sensitive data or corruption of the user interface.
Suggested Countermeasures
It is recommended to: • Filter input on arrival • Encode data on output • Use appropriate response headers • Use Content Security Policy (CSP) to reduce the severity of any existing XSS vulnerabilities.
References
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp https://caniuse.com/#feat=contentsecuritypolicy https://content-security-policy.com/

Proof of concept:



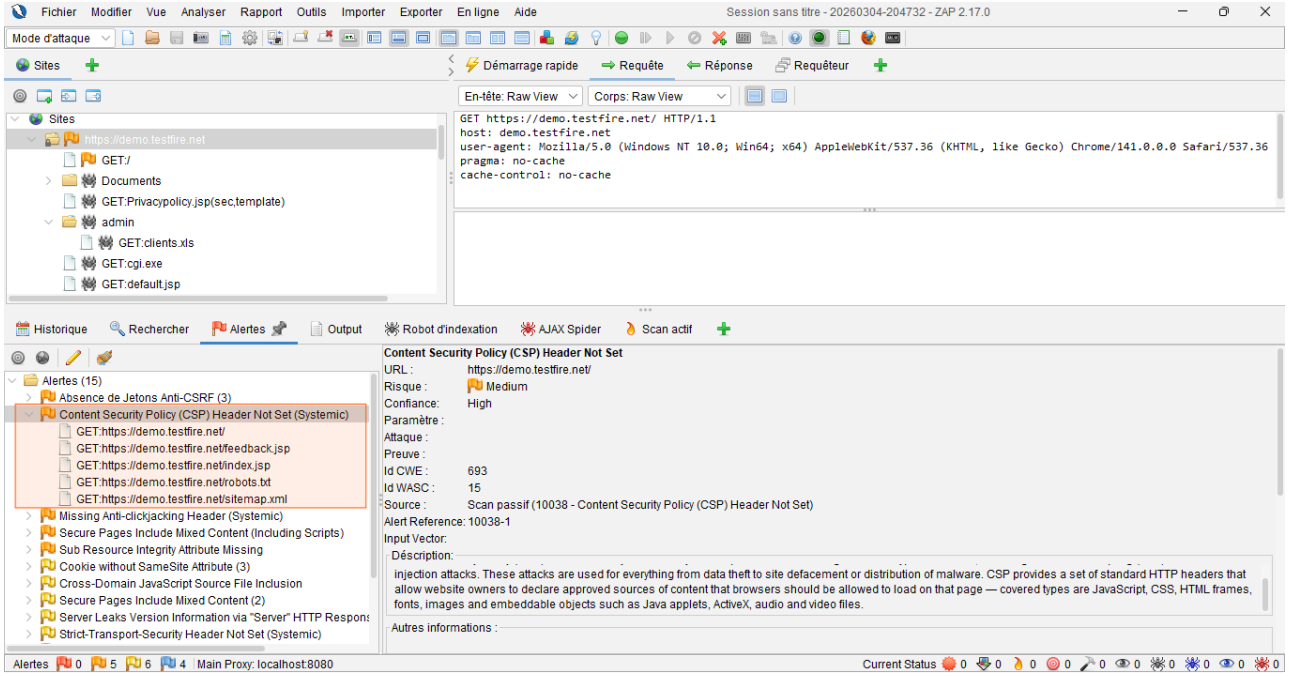


Fig 2: Content Security Policy (CSP) Header Not Set

3. Missing Anti-clickjacking Header

Reference No:	**Risk Rating:**
WEB_VUL_03	**Medium**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
The server does not send the necessary instructions to prevent the site from being displayed within a frame on another malicious site.	
Vulnerability Identified by / How It Was Discovered	
Automated analysis /Manual Analysis	
Vulnerable URLs / IP Address	
http://demo.testfire.net/	
Implications / Consequences of not Fixing the Issue	
• The absence of CSP allows malicious scripts to be executed that can read data entered by customers (logins, account numbers) and send it to a server controlled by the attacker. • An attacker can steal users' session cookies to impersonate them without needing their password. • Due to the clickjacking vulnerability and the absence of anti-CSRF tokens, a user can validate bank transfers or profile changes without their knowledge simply by browsing another compromised page. • A hacker can modify the legitimate display of the site to deceive users (for example, displaying a fake message asking them to re-enter their credit card details).	
Suggested Countermeasures	

- If the page should never be embedded in a frame: Use the DENY directive for the X-Frame-Options header or frame-ancestors 'none' for CSP. This blocks any attempt to display the site via a **<frame>** or **<iframe>** tag.

- If the page should only be embedded by your own server: Use the value **SAMEORIGIN**. This allows the use of frames (**FRAMESET**) only if the parent site belongs to the same domain as the application.

- Prefer the implementation of the frame-ancestors directive within the Content Security Policy, as it offers finer and more robust control than the old X-Frame-Options header.

References

https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options

Proof of concept:

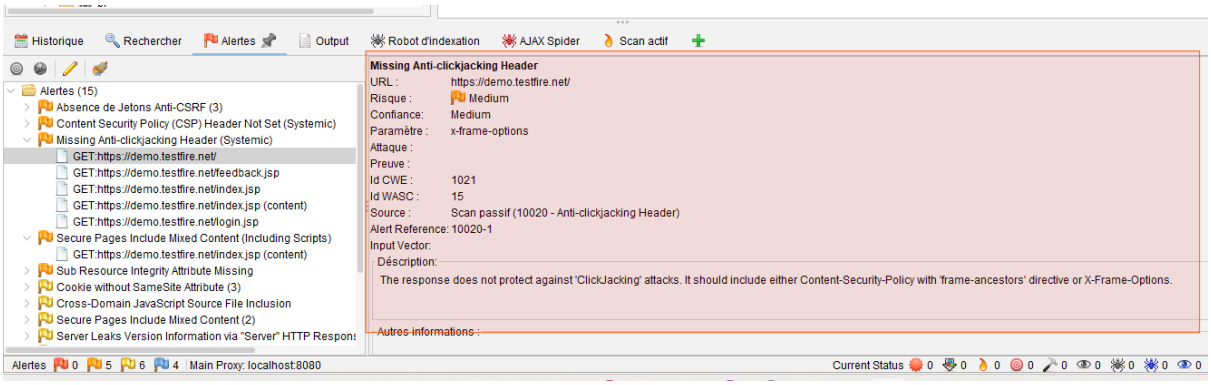


Fig 3: Missing Anti-clickjacking Header

4. Secure Pages Include Mixed Content (Including Scripts)

Reference No:	**Risk Rating:**
WEB_VUL_04	**Medium**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
The page includes mixed content, that is content accessed via HTTP instead of HTTPS.	
Vulnerability Identified by / How It Was Discovered	
Automated Analysis / Manual Analysis	
Vulnerable URLs / IP Address	
https://demo.testfire.net/index.jsp?content=personal_investments.htm	
Implications / Consequences of not Fixing the Issue	
• The overall security of the page is reduced to the level of the least secure element. If a script is loaded via HTTP, an attacker can modify it during transfer to take complete control of the secure page.	

- Risk of a man-in-the-middle attack: A hacker on the same network (e.g., public Wi-Fi) can intercept and modify HTTP resources to inject malicious code or false information into the Altoro Mutual website.

- Loss of browser trust: Modern browsers often display an alert ("Locked padlock" or "Not secure") or block the content altogether. For a bank, seeing a security alert message on its account is catastrophic for its credibility.

- Data leak: Sensitive information passing through the page could be exposed via these unencrypted requests, even if the main URL is HTTPS.

Suggested Countermeasures

- A page that is available over SSL/TLS must be comprised completely of content which is transmitted over SSL/TLS.
- The page must not contain any content that is transmitted over unencrypted HTTP.
- This includes content from third party sites.

References

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Protection_Cheat_Sheet.html

5. Sub Resource Integrity Attribute Missing

Reference No:	**Risk Rating:**
WEB_VUL_05	**Medium**

Tools Used:

OWASP ZAP

Vulnerability Description:

The application uses scripts or style sheets hosted on external servers (such as CDNs) without checking whether these files have been modified without your knowledge.

Vulnerability Identified by / How It Was Discovered

Automated Analysis / Manual Analysis

Vulnerable URLs / IP Address

https://demo.testfire.net/index.jsp?content=personal_investments.htm

Implications / Consequences of not Fixing the Issue

- Malware injection: A single modification to an external script can compromise the security of all your users simultaneously.
- Loss of control: Your security depends on the security of a third party over which you have no control.

Suggested Countermeasures

- Provide a valid integrity attribute to the tag. Encode data on output

References

https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity

Proof Of Concept:

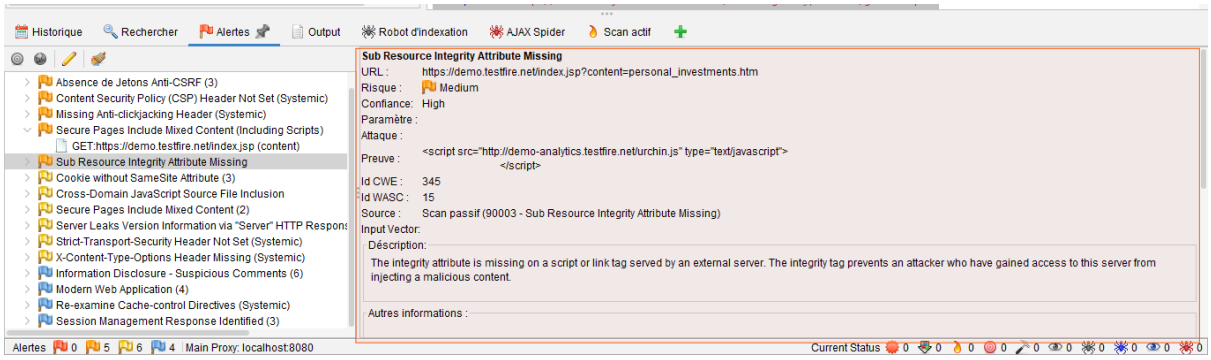


Fig 4: Sub Resource Integrity Attribute Missing

6. Cookie without SameSite Attribute

Reference No:	**Risk Rating:**
WEB_VUL_06	**Low**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis	
Vulnerable URLs / IP Address	
http://testphp.vulnweb.com/guestbook.php	
Implications / Consequences of not Fixing the Issue	
• **CSRF (Cross-Site Request Forgery) facilitation**: Without the SameSite attribute, an attacker can force your browser to send your session cookies to Altoro Mutual from a booby-trapped site. This is the direct complement to the CSRF vulnerability we saw earlier. • **Inclusion of malicious scripts (XSSI)**: A third-party site could attempt to load private resources from your bank account by taking advantage of the fact that the browser automatically sends your session credentials. • **Timing attacks**: Malicious sites can guess whether or not you are logged into your bank by measuring the server's response time, which compromises your privacy.	
Suggested Countermeasures	
• Ensure that the SameSite attribute is set to either 'lax' or ideally 'strict' for all cookies.	
References	

https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-cookie-same-site

Proof of Concept:

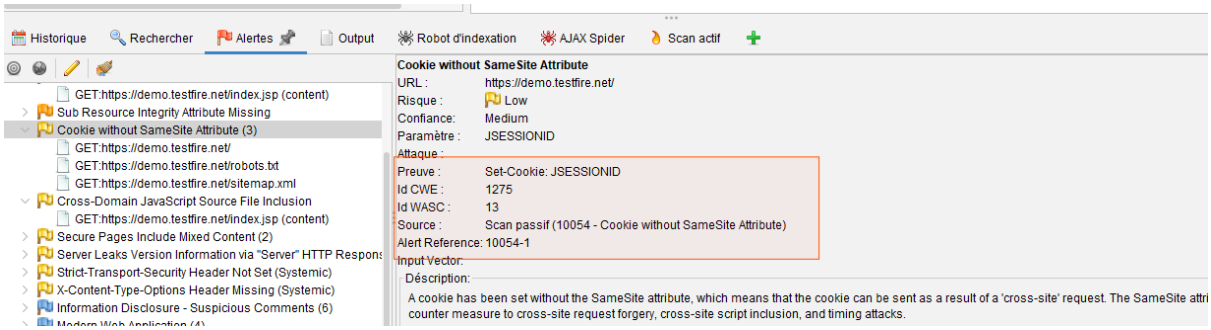


Fig 5: Cookie without SameSite Attribute

7. Cross-Domain JavaScript Source File Inclusion

Reference No:	**Risk Rating:**
WEB_VUL_07	**Low**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
The page includes one or more script files from a third-party domain.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis	
Vulnerable URLs / IP Address	
https://demo.testfire.net/index.jsp?content=personal_investments.htm	
Implications / Consequences of not Fixing the Issue	
• **Supply Chain Attack**: If the third-party server is hacked, the attacker can modify the script to steal the credentials of all Altoro Mutual customers. You have no control over the security of this external server. • **Spying and Data Leaks (Privacy):** Third-party scripts (including advertising or analytics trackers) can collect data on your users' behavior without your explicit consent, which raises GDPR compliance issues. • **Single Point of Failure (SPOF):** If the third-party server goes down or slows down, your application may become unavailable or very slow, as the browser waits to load this external script to display the page.	
Suggested Countermeasures	
• Ensure JavaScript source files are loaded from only trusted sources, and the sources can't be controlled by end users of the application.	

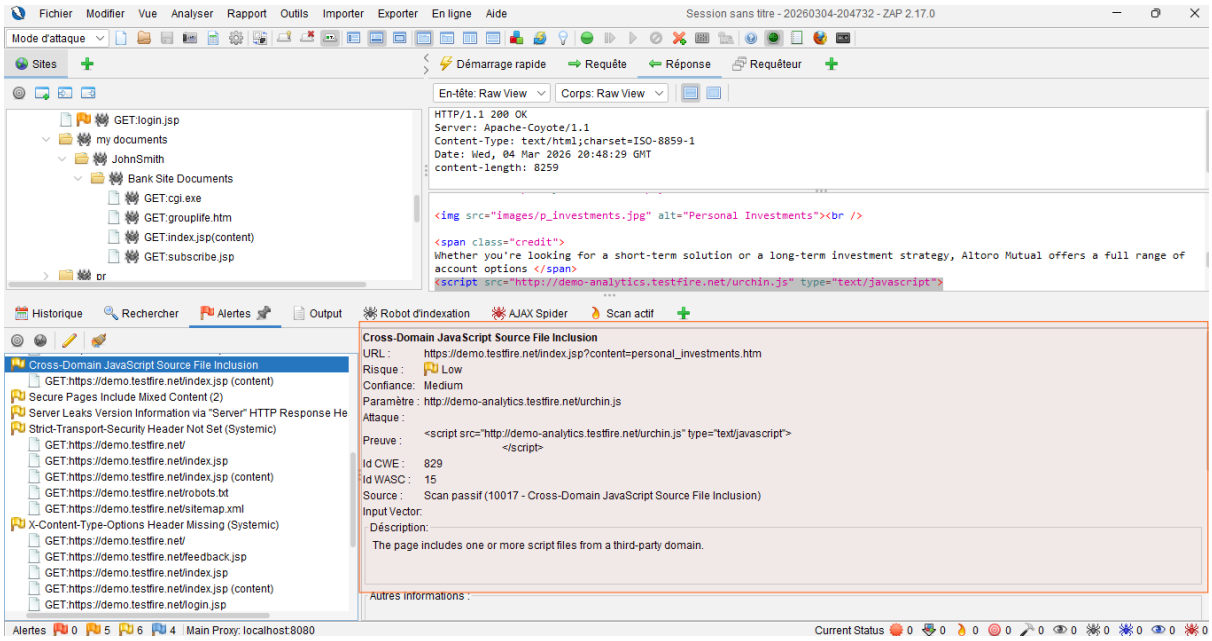


Fig 6: Cross-Domain JavaScript Source File Inclusion

8. X-Content-Type-Options Header Missing

Reference No:	**Risk Rating:**
WEB_VUL_08	**Low**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
The browser ignores the server tag and tries to guess what is in the file on its own.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis	
Vulnerable URLs / IP Address	
https://demo.testfire.net/	
Implications / Consequences of not Fixing the Issue	
• **Malicious code execution:** A harmless text file can be interpreted as an executable script (XSS). • **Bypassing security filters:** Protections that only check file extensions (.jpg, .txt) become useless if the browser decides to change the file type when reading.	
Suggested Countermeasures	
• Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. • If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.	

References
https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers

9. Information Disclosure - Suspicious Comments

Reference No:	Risk Rating:
WEB_VUL_09	**Info**
Tools Used:	
OWASP ZAP	
Vulnerability Description:	
The response appears to contain suspicious comments which may help an attacker.	
Vulnerability Identified by / How It Was Discovered	
Manual Analysis	
Vulnerable URLs / IP Address	
https://demo.testfire.net/	
Implications / Consequences of not Fixing the Issue	
• **Reconnaissance Assistance:** This is a gold mine for the preparation phase of an attack. The more a hacker knows about your system, the more precise and effective their attacks will be. • **Information Leakage:** This can reveal structural weaknesses that the attacker would never have been able to guess otherwise. • **Evidence of Negligence:** The presence of development comments in production shows a lack of rigor in the deployment process.	
Suggested Countermeasures	
• Remove all comments that return information that may help an attacker and fix any underlying problems they refer to	

----------EOF----------